

Mini Core Banking Transaction Processing System

Project Understanding Document | Oracle SQL & PL/SQL

1. [OVERVIEW] Project Overview

This project simulates a **real-world core banking system** similar to what banks use internally (such as Oracle Flexcube). Built using **Oracle SQL and PL/SQL**, it focuses on handling financial transactions safely and consistently.

2. [OBJECTIVE] Project Objective

- Manage bank accounts
- Perform deposits and withdrawals
- Enable fund transfers between accounts
- Maintain transaction history
- Ensure data consistency (no money loss or duplication)
- Handle errors safely
- Track all changes for audit purposes

3. [CONCEPT] Core Banking Concept

KEY RULE: *Money should never be lost or duplicated*

Every transaction must follow: **safe execution, proper validation, full tracking, and rollback on failure.**

4. [FLOW] How a Real Bank Transaction Works

Example: Transfer Rs. 1000 from Account A to Account B

1	Validate both accounts exist
2	Check sender has sufficient balance
3	Lock both accounts (prevent concurrent issues)
4	Deduct money from sender
5	Add money to receiver
6	Record the transaction
7	Commit changes

IMPORTANT: *If any step fails, the entire transaction is rolled back.*

5. [ARCHITECTURE] System Architecture

User Layer	PL/SQL Logic Layer	Database Layer
------------	--------------------	----------------

-> User Layer

Initiates actions such as deposit, withdraw, and transfer.

-> PL/SQL Logic Layer

Contains all procedures; handles validations, transactions, and errors.

-> Database Layer

Stores all persistent data including accounts, transactions, and logs.

6. [DATABASE] Database Components

Accounts Table	Account ID, Customer Name, Balance, Account Type, Status
Transactions Table	Deposits, Withdrawals, Transfers
Audit Logs Table	Old Balance, New Balance, Timestamp
Error Logs Table	Error Message, Procedure Name, Account Involved

7. [FUNCTIONS] Core Functionalities

Account Creation	Create a new bank account with all required details.
Deposit	Add money into an account with full validation and logging.
Withdrawal	Deduct money only after confirming sufficient balance.
Fund Transfer	Transfer funds between accounts atomically with locking.

8. [ACID] Transaction Management — ACID Properties

Atomicity	All steps succeed or fail together
Consistency	Data remains correct at all times
Isolation	Concurrent transactions do not interfere
Durability	Changes are permanent after commit

9. [CONCURRENCY] Concurrency Control

To prevent double-spending and race conditions, the system uses:

- Row-level locking via **SELECT FOR UPDATE** — prevents two transactions from modifying the same account simultaneously.
- Transactions wait if a target row is already locked by another session.
- Consistent lock ordering across all procedures to prevent deadlocks.

10. [SAFETY] Transaction Safety

SAVEPOINT	Acts as a checkpoint within a transaction. Allows partial rollback without discarding all changes.
ROLLBACK	Reverts all changes back to the last SAVEPOINT or to the start of the transaction on any failure.
COMMIT	Permanently saves all changes once every step of the transaction completes successfully.

11. [EXCEPTIONS] Exception Handling

The system handles the following error conditions:

- Invalid account number
- Insufficient balance
- Invalid or null input
- Unexpected system failures

Custom exceptions are raised using:

```
RAISE_APPLICATION_ERROR ( error_number, error_message )
```

12. [AUDIT] Audit System

A database **TRIGGER** fires automatically on every account balance change, recording the complete audit trail in the Audit Logs table:

- Old balance before the transaction
- New balance after the transaction
- Timestamp of every change

This ensures: **full transparency**, **fraud detection**, and **regulatory compliance**.

13. [ERROR LOG] Error Logging System

Error logging uses an **autonomous transaction** so that even when the main transaction rolls back, the error record is permanently saved:

```
PRAGMA AUTONOMOUS_TRANSACTION
```

This guarantees that every failed operation leaves a trace, enabling post-failure diagnosis.

14. [REPORTS] Reporting System

Using PL/SQL cursors, the system generates the following operational reports:

Report	Description
Daily Transaction Summary	All deposits, withdrawals, and transfers processed on a given day.
Failed Transaction Report	All transactions that were rolled back along with the error reason.
Top Accounts Report	Accounts ranked by current balance for management review.

15. [TESTING] Testing & Validation

The complete system is tested across the following scenarios:

Normal Operations	Standard deposit, withdrawal, and transfer flows
Invalid Inputs	Null values, negative amounts, wrong account IDs
Insufficient Balance	Withdrawal or transfer exceeding available funds
Concurrent Transactions	Two sessions updating the same account simultaneously
System Failures	Simulated mid-transaction crashes and rollback verification

16. [HIGHLIGHTS] Key Highlights

+ Real-world banking logic
+ ACID-compliant transactions
+ Row-level locking
+ Audit trail implementation
+ Autonomous error logging
+ Production-style exception handling

17. [CONCLUSION] Conclusion

This project demonstrates how a **core banking system ensures safe, consistent, and reliable transaction processing** using PL/SQL. It reflects real-world practices used in production banking systems and prepares the developer for enterprise-level application development.